

潜在変数からの RSRP エピソード生成

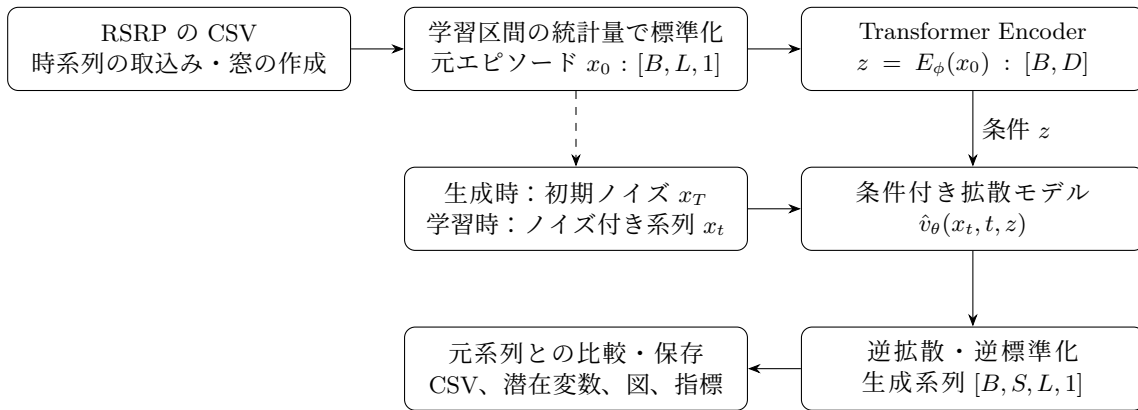
EpisodicDT-small：プログラム説明資料

対象形式：direct_z_episode_v2 作成日：2026 年 9 月 15 日

1 目的と全体構成

本プログラムの現在の目的は、元エピソードから抽出した潜在変数を用いて、元データと似た特徴を持つ新しいエピソードを生成できるか確認することである。入力履歴に続く未来だけを予測する方式から、エピソード全体を入力・生成対象とする方式へ変更した。対象は単変量の RSRP 系列であり、通信制御 AI や物理シミュレータとの接続は現行処理に含まない。

1.1 全体のアーキテクチャ



破線は学習時に正解系列へノイズを加える経路を示す。生成時は**潜在変数 z** と**拡散ノイズ**だけを使い、元エピソードを生成器へ直接渡さない。同じ z からでも、ノイズを変えることで複数系列を生成する。

1.2 入出力と初期設定

項目	内容
入力データ	CSV の <code>rsrp</code> 列。単位 dBm。窓はファイル境界をまたがない。
エピソード長	$L = 40$ ステップ。約 2 Hz なら約 20 秒だが、実時間は元の測定間隔に依存する。
潜在変数	$D = 32$ 次元の実数ベクトル。Encoder の出力そのもの。
生成数	標準評価は元エピソードごとに $S = 32$ 系列。
出力データ	生成系列・元系列の CSV、潜在変数ファイル、評価 JSON、比較図。
対象外	多変量観測、可変長系列、基地局操作による状態遷移の計算。

B はバッチ内のエピソード数である。現在は通常・危険の全窓を扱い、危険だけへの自動抽出や危険例生成率による評価は行わない。

2 データ処理とモデルの内部構造

2.1 窓の作成と標準化

RSRPEpisodeDataset はファイルごとに長さ L の連続窓を 1 ステップ刻みで作る。長さ N のファイルでは $N - L + 1$ 個の窓が得られる。短すぎるファイルは窓作成対象から除かれる。入力はある値の RSRP 系列を想定し、非有限値があればエラーとする。時刻の補間を行わないため、時刻間隔や欠測の処理は前処理側で確認する必要がある。

学習用の各系列を時間順に前方 80%・後方 20% へ分け、それぞれ学習・検証に使う。境界をまたぐ窓は除外する。正規化の平均 m_{tr} と標準偏差 s_{tr} は **検証部分を除いた学習区間のみ**から求める。

$$x_{0,\ell} = \frac{r_\ell - m_{\text{tr}}}{s_{\text{tr}}}, \quad \tilde{r}_\ell = s_{\text{tr}}\tilde{x}_{0,\ell} + m_{\text{tr}}.$$

標準偏差には数値安定性のため下限 10^{-6} を置く。元エピソードごとの平均値や最終値を生成結果へ加算する処理はない。最終評価には別途指定された評価用ファイルを用いる。

2.2 Encoder：全体系列から直接 z を得る

処理	内容	標準のテンソル形状
入力射影	RSRP の 1 次元を 128 次元へ線形変換	$[B, 40, 1] \rightarrow [B, 40, 128]$
位置情報	正弦・余弦による位置符号化を加算	$[B, 40, 128]$
Transformer	4 ヘッド、3 層の Encoder	$[B, 40, 128]$
代表特徴の抽出	最後のトークンの出力を使用	$[B, 128]$
潜在射影	線形層と SiLU	$[B, 128] \rightarrow [B, 32]$

Attention には因果マスクを用いず、最後のトークンの表現もエピソード全体を参照できる。最後の RSRP 値だけを使うという意味ではない。

$$z = E_\phi(x_0)$$

$\mu \cdot \sigma$ の推定、潜在変数の確率的サンプリング、KL 正則化は行わない。学習時には dropout（設定値 0.05）を用いるが、推論時に `model.eval()` とすると、同じ入力から同じ z を得る。各潜在次元に遮蔽量などの物理的意味を指定してはいない。

2.3 拡散モデル： (x_t, t, z) から v を予測する

拡散器のクラス内では NoisePredictor という名称を使っているが、現在の出力対象はノイズそのものではなく速度パラメータ v である。ノイズ付き系列 x_t を平坦化した L 次元、潜在変数 z の 32 次元、拡散ステップ t の 128 次元埋込みを連結する。標準構成では入力は $40 + 32 + 128 = 200$ 次元で、**200→128→128→40** の MLP（中間層に SiLU）から $[B, 40, 1]$ を出力する。この t はエピソード中の観測時刻とは別の、ノイズ除去のステップ番号である。

3 学習・生成アルゴリズム

3.1 学習時の計算

数式では拡散ステップを $t = 1, \dots, T$ と表す（コードの配列添字は $0, \dots, T-1$ ）。 $T = 100$ で、累積信号係数 $\bar{\alpha}_T = 0$ になるようノイズスケジュールを再調整する。各エピソードについて t を一様に選び、独立な $\epsilon \sim \mathcal{N}(0, I)$ を生成する。

$$z = E_\phi(x_0), \quad (1)$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad (2)$$

$$v_t = \sqrt{\bar{\alpha}_t} \epsilon - \sqrt{1 - \bar{\alpha}_t} x_0, \quad (3)$$

$$\mathcal{L}(\phi, \theta) = \mathbb{E}_{x_0, t, \epsilon} \left[\frac{1}{L} \|\hat{v}_\theta(x_t, t, E_\phi(x_0)) - v_t\|_2^2 \right]. \quad (4)$$

損失は v 予測 MSE のみである。正解の z を与える教師データはなく、生成に役立つ表現を学ぶよう、 z を通じて Encoder にも勾配を伝える。現在は入力と生成対象が同じエピソードである。モデル API には同じ環境条件から得た別エピソードを `target` として渡す機能もあるが、標準の学習ループでは使用していない。

学習設定	現在の設定値・処理
最適化	AdamW、学習率 10^{-4} 、バッチサイズ 64。
安定化	勾配ノルムの上限 1.0。
学習回数	最大 150 エポック、最低 30 エポック。
モデル選択	検証 v 予測 MSE の実際の最小値を保存。
Early Stopping	0.001 を超える検証損失改善が 15 エポック連続でなければ、最低回数以降に終了。
再現性	学習 seed は 42。検証 seed は 12345 で毎回固定し、学習側の乱数状態を保存・復元。

ログの `loss` と `diffusion_mse` は同じ値である。CRPS は診断用に記録するが、現行のモデル選択には使っていない。

3.2 生成時の計算

元エピソードを一度 Encoder へ入力して z を得る。各生成例の x_T は独立な標準正規ノイズから開始し、 z を固定して逆拡散する。

$$\hat{x}_0 = \sqrt{\bar{\alpha}_t} x_t - \sqrt{1 - \bar{\alpha}_t} \hat{v}_\theta(x_t, t, z), \quad (5)$$

$$x_{t-1} = c_t \hat{x}_0 + d_t x_t + \sqrt{\tilde{\beta}_t} \xi_t. \quad (6)$$

$c_t, d_t, \tilde{\beta}_t$ はノイズスケジュールから計算する DDPM の係数であり、 ξ_t は新しい標準正規ノイズである。最後のステップではノイズを加えない。完了後、学習時の統計量を用いて dBm へ戻す。初期ノイズと途中のノイズが異なるため、同じ z から複数の系列が得られる。

3.3 API と保存済み z の利用

```
model.eval()
z = model.encode(normalized_episode)          # [B,32]
x = model.generate(z, num_samples=32)         # [B,32,40,1]
rsrp_dbm = x * training_std + training_mean
```

保存済み z を使う場合は Encoder と元 CSV を参照せず生成できる。モデルと潜在ファイルの対応はチェックポイントの SHA-256 で照合する。旧確率潜在モデルの重み・潜在ファイルとは非互換である。

4 評価方法・出力・利用手順

4.1 評価指標の定義

元系列の値を $y_{n\ell}$ 、生成値を g_{nsl} とし、 n をエピソード、 $s = 1, \dots, S$ を生成例、 ℓ をステップとする。生成平均は $\bar{g}_{n\ell} = S^{-1} \sum_s g_{nsl}$ である。

$$\text{MAE} = \frac{1}{NL} \sum_{n,\ell} |\bar{g}_{n\ell} - y_{n\ell}|,$$
$$\text{CRPS} = \frac{1}{NL} \sum_{n,\ell} \left[\frac{1}{S} \sum_s |g_{nsl} - y_{n\ell}| - \frac{1}{2S^2} \sum_{s,r} |g_{nsl} - g_{nr\ell}| \right].$$

いずれも RSRP の差に基づく量で、小さいほど元系列に整合する。

その他の指標	定義・注意点
90% 区間被覆率	各点の元の値が生成値の 5–95% 分位区間に入る割合。区間幅と併せて評価する。
ステップ別統計	CRPS、MAE、平均誤差、元・生成の標準偏差、被覆率、区間幅。
固定 z 内の標準偏差	同じ元エピソード・同じステップの生成例間標準偏差を、全エピソード・ステップで平均する。

これらは元エピソード全体を Encoder に与えた条件付き再生成の評価であり、独立した未来予測の精度ではない。生成のばらつきが大きければよいわけではなく、過大な裾や区間幅にも注意する。危険例生成率は現段階の指標に含まない。

4.2 図と出力ファイル

図と指標の保存先は `results/figures/direct_z_episode_v2/` である。

代表系列の図 `episodes.png`、全値の箱ひげ図 `episode_boxplot.png`、ステップ別の図 `episode_boxplot_by_step.png`、指標 `episode_metrics.json` を保存する。箱は 25–75% 点、中央線は中央値、ひげは 1.5 IQR 以内の最遠点であり、外れ値は非表示である。ひげを最大・最小値と解釈しない。生成側は元データの S 倍の標本数を持つ。系列比較図では緑が元系列、青が生成例のステップ別中央値、濃い帯が 25–75% 点、薄い帯が 5–95% 点である。青線は一つの生成例そのものではない。帯は点ごとの区間で、系列全体の被覆確率を表さない。図の median MAE は中央値との誤差であり、評価 JSON の MAE は生成平均との誤差である。

生成 CSV の列は `episode_id`, `sample_id`, `episode_step`, `rsrp_generated_dbm`。元系列 CSV、 z と元ファイル・窓開始位置を含む潜在ファイルも保存する。チェックポイントには重み、設定、標準化統計量、最良エポックなどを保存する。

4.3 実行方法とコードの対応

プロジェクト `EpisodicDT-small` 直下で実行する。

```
conda activate episodicdt
python -m src.train.train --config configs/config.yaml
python -m src.evaluation.evaluate --samples 32 --seed 12345
# 保存済みzからの生成（各パスは実際の保存先を指定）
python -m src.evaluation.generate --checkpoint MODEL.pt \
    --latents LATENTS.pt --output generated.csv --samples 16
```

`src/data/dataset.py` がデータ、`src/models/episodic_diffusion.py` がモデル、`src/train/` が学習・検証、`src/evaluation/` が評価・再生成を担当する。設定は `configs/config.yaml` で管理する。

4.4 現在の限界と研究への位置付け

小さい再生成誤差だけでは、元データの記憶やコピーを排除できない。潜在変数の交換実験、最近傍系列との比較、時間構造・分位点の比較が今後必要になる。物理的な潜在状態との対応や生成例の実現可能性も未検証である。最終的には潜在表現からシミュレータの環境条件を生成し、制約と実測整合性を確認した上で、そのシナリオを用いた制御 AI 学習の効果を独立したシミュレーション条件で評価する。